

% 1. Stergerea ultimului element dintr-o lista incompleta

length(L,0):-var(L),!.

length([H|T],Rez):-length(T,Rez1),

Rez is Rez1+1.

delete_last([H|T],T):-var(T),!.

delete_last([H|T],[H|R]):-delete_last(T,R).

% 2. Stergerea unui element dintr-o lista completa

delete_last_com([H|[]],[]).

delete_last_com([H|T],[H|R]):-delete_last_com(T,R).

delete_last_com([],[]).

% 3. Stergerea unui element dintr-o lista diferentaS

add(X,LS,LE,RS,RE):-RS=LS,LE=[X|RE].

stergere([H],LE,RE,RE).

stergere([H|T],LE,[H|RS],RE):-stergere(T,LE,RS,RE).

% 4. Concatenarea a doua liste diferenta

```
append_dif([],[],YS,YE,YS,YE).
```

```
append_dif([H|T],XE,YS,YE,[H|RS],RE):-append_dif(T,XE,YS,YE,RS,RE).
```

% 5. Concatenare liste incomplete

```
append_li(M,L,L):-var(M),!.
```

```
append_li([H|T],L,[H|R]):-append_li(T,L,R).
```

% 6. Lista completa in lista differenta

```
transf_ld([],RE,RE).
```

```
transf_ld([H|T],[H|RS],RE):-transf_ld(T,RS,RE).
```

% 7. Lista completa in lista incompleta

```
ld_lc([],RE,[]).
```

```
ld_lc([H|T],RE,[H|R]):-ld_lc(T,RE,R).
```

% 8. Lista differenta in lista completa

```
dl_cl([],L,[]):-!.
```

```
dl_cl([X|T],L,[X|R]):-dl_cl(T,L,R).
```

% 9. Lista incompleta in lista completa

il_cl(T,[]):-var(T),!.

il_cl([X|T],[X|R]):-il_cl(T,R).

% 10. Lista incompleta in lista diferenta

li_lc(T,[]):-var(T).

li_lc([H|T],[H|R]):-li_lc(T,R).

% 11. Lista diferenta in lista incompleta

ld_li([],RE,T).

ld_li([H|T],RE,[H|R]):-ld_li(T,RE,R).

% Lista completa in lista incompleta

transf_li([],L).

transf_li([H|T],[H|R]):-transf_li(T,R).

%transformare liste incompleta in lista diferenta

li_ld(T,RE,RE):-var(T),!.

li_ld([H|T],[H|RS],RE):-li_ld(T,RS,RE).

% 12. Inversarea unei liste incomplete.

inversare_li(T,T):-var(T),!.

```
inversare_li([H|T],Rez):-inversare_li(T,Rez1),  
    append_li(Rez1,[H|_],Rez).
```

% 13. Inversarea unei liste diferenta.

```
inversare_df([],LE,RE,RE).  
  
inversare_df([H|T],LE,RS,RE):-inversare_df(T,LE,RS1,RE1),  
    append_dif(RS1,RE1,[H|LY],LY,RS,RE).
```

% 14. Inversarea unei liste complete

```
inversare([H|T],Rez):-inversare(T,Rez1),  
    append(Rez1,[H],Rez).
```

```
inversare([],[]).
```

% 15. Concatenarea a doua liste complete

```
append([],L,L).  
  
append([H|T],L,[H|R]):-append(T,L,R).
```

% 16. Gasirea elementului max dintr-o lista cu recursivitate inapoi

```
maxim_inapoi([H],H).  
  
maxim_inapoi([H|T],H):-maxim_inapoi(T,M),H>M.  
  
maxim_inapoi([H|T],M):-maxim_inapoi(T,M).
```

% 17. Gasirea elementului max dintr-o lista cu recursivitate inainte

```
maxim([],Max,Max).
```

```
maxim([H|T],Max,Maxp):-H>Maxp,!,
```

```
    maxim(T,Max,H).
```

```
maxim([H|T],Max,Maxp):-maxim(T,Max,Maxp).
```

```
maximul([H|T],Max):-maxim([H|T],Max,H).
```

% 18. Se da o lista. Sa se descomp in 2 liste dupa un pivot P dat.

```
inlocuire_pivot(_,[],[],[]).
```

```
inlocuire_pivot(P,[H|T],[H|Mic],Mare):-H<P,!,
```

```
    inlocuire_pivot(P,T,Mic,Mare).
```

```
inlocuire_pivot(P,[H|T],Mic,[H|Mare]):-inlocuire_pivot(P,T,Mic,Mare).
```

% 19. Folosind for cu decrementare sa se genereze o lista de genul [N,...,1] cu N da

```
for(0,[]).
```

```
for(I,[I|R]):-I>0,
```

```
    Ni is I-1,
```

```
    for(Ni,R).
```

% 23 Inlocuirea tuturor aparitiilor unui element cu un alt element

```
replace_all(Old,New,[Old|T],[New|R]):-replace_all(Old,New,T,R).
```

```
replace_all(Old,New,[H|T],[H|R]):-replace_all(Old,New,T,R).
```

```
replace_all(_,_,[],[]).
```

%24 Inlocuirea primei aparitii a unui element cu un alt element

```
replace_one(Old,New,[Old|T],[New|T]).
```

```
replace_one(Old,New,[H|T],[H|R]):-replace_one(Old,New,T,R).
```

```
replace_one(_,_,[],[]).
```

% 25. Inlocuirea primei aparitii a unui element U cu un element N dintr-o lista diferenta

```
replace_ld(_,_,[],L,RF,RF).
```

```
replace_ld(Old, New,[Old|T],L,[New|T],L):-!.
```

```
replace_ld(Old,New,[H|T],L,[H|RF],RL):-replace_ld(Old,New,T,L,RF,RL).
```

% 26. Inlocuirea aparitiilor a unui element U cu un alt element N dintr-o lista incompleta

```
replace_li(_,_ ,T,T):-var(T),!.
```

```
replace_li(Old,New,[Old|T],[New|R]):-replace_li(Old,New,T,R).
```

```
replace_li(Old,New,[H|T],[H|R]):-replace_li(Old,New,T,R).
```

% 27. Generare lista de elem de la 1 la N

```
for_1_n(0,L,L).
```

```
for_1_n(Index,L,R):-Index>0,
```

```
NouIndex is Index-1,  
for_1_n(NouIndex,[Index|L],R).
```

```
for_1n(N,R):-for_1_n(N,[],R).
```

% 28. Interclasare a doua liste ordonate

```
interclasare_liste(L1,[],L1).  
interclasare_liste([],L2,L2).  
interclasare_liste([H1|T1],[H2|T2],[H1|R]):-H1<H2,!,  
                                                    interclasare_liste(T1,[H2|T2],R).  
interclasare_liste(L1,[H2|T2],[H2|R]):-interclasare_liste(L1,T2,R).
```

%Alte exemple

%29. Sa se stearga toate aparitiile unui element dat dintr-o lista incompleta

```
delete_all_li(_,T,T):-var(T),!.  
delete_all_li(X,[X|T],R):-delete_all_li(X,T,R).  
delete_all_li(X,[H|T],[H|R]):-delete_all_li(X,T,R).
```

% 30. Sa se elimine dublurile dintr-o lista incompleta

%varianta care il lasa pe primul

```
member_li(_,L):-var(L),!,fail.
```

member_li(X,[X|T]):-!.

member_li(X,[H|T]):-member_li(X,T).

remove_dub_li(T,T):-var(T),!.

remove_dub_li([H|T],R):-member_li(H,T),

remove_dub_li(T,R).

remove_dub_li([H|T],[H|R]):-remove_dub_li(T,R).

%31. varianta care il lasa pe al doilea

remove_dub(T,T):-var(T),!.

remove_dub([H|T],[H|R]):-delete_all_li(H,T,R1),

remove_dub(R1,R).

% 32 Se da o lista de numere intregi, sa se afiseze sublistele care au aceeasi monotonie

ordonat([H]).

ordonat([H1,H2|T]):-H1<H2, !,

ordonat([H2|T]).

subliste([],[]).

subliste([H|T],[H|R]):-ordonat(H),!,

```
subliste(T,R).
```

```
subliste([H|T],R):-subliste(T,R).
```

% 33. Generare aranjamente de lungime k a unei liste

```
element([H|T],H,T).
```

```
element([H|T],C,[H|R]):-element(T,C,R).
```

```
permutari([],[]).
```

```
permutari(L,[E|C]):-element(L,E,X),
```

```
permutari(X,C).
```

```
tree1(t(6, t(4, t(2, nil, nil)), t(5, nil, nil)), t(9, t(7, nil, nil), nil))).
```

```
tree2(t(8, t(5, nil, t(7, nil, nil)), t(9, nil, t(11, nil, nil)))).
```

```
tree3(t(7, t(5, t(3, _, _), t(6, _, _)), t(11, _, _))).
```

```
tree4(t(7, t(5, t(3, _, _), t(6, _, _)), t(11, _, t(12, _, _)))).
```

```
%Arbori
```

% 34. Inserare in arbore binar de cautare

```
inserare_arbore(Cheie,nil,t(Cheie,nil,nil)):-write('Inserted '), write(Key), nl.
```

```
inserare_arbore(Cheie,t(Cheie,L,R),t(Cheie,L,R)):-write('Cheie exista '), nl.
```

```
inserare_arbore(Cheie,t(Key,L,R),t(Key,NL,R)):-Cheie<Key,!,
```

```
inserare_arbore(Cheie,L,NL).
```

```
inserare_arbore(Cheie,t(Key,L,R),t(Key,L,NR)):-inserare_arbore(Cheie,R,NR).
```

% 35.a Cautare cheie intr-un arbore terminat in variabila

```
cautare_arbore_in(_,T):-var(T),!,fail.
```

```
cautare_arbore_in(Cheie, t(Cheie,_,_)):-!.
```

```
cautare_arbore_in(Cheie,t(Key,L,_)):-Cheie<Key,!,
```

```
cautare_arbore_in(Cheie,L).
```

```
cautare_arbore(Cheie,t( _,_,R)):-cautare_arbore_in(Cheie,R).
```

% 36.b Cautare cheie intr-un arbore simplu

```
cautare_arbore(Cheie, t(Cheie,_,_)):-!.
```

```
cautare_arbore(Cheie,t(Key,L,_)):-Cheie<Key,!,
```

```
cautare_arbore(Cheie,L).
```

```
cautare_arbore(Cheie,t( _,_,R)):-cautare_arbore(Cheie,R).
```

% 37. Inserare cheie intr-un arbore incomplet

```
inserare_arbore_in(Cheie,t(Cheie,_,_)):-write('Inserted '), write(Cheie), nl.
```

```
inserare_arbore_in(Cheie,t(Key,L,R)):-Cheie<Key,!,
```

```
inserare_arbore_in(Cheie,L).
```

```
inserare_arbore_in(Cheie,t(_,_,R)):-inserare_arbore_in(Cheie,R).
```

% 38. Cozi cu liste diferenta + exemplificare operatii

```
enqueue(X,F,[X|L],F,L). % enqueue(5,[1,2,3,4|L],L,FR,LR). adauga in coada  
deque(H,[H|F],L,F,L). % deque(H,[1,2,3,4|L],L,FR,LR). scoate din coada
```

% 39. Stiva cu liste diferenta

```
enstiva(X,F,L,[X|F],L). % enstiva(5,[1,2,3,4|L],L,FR,LR).  
destiva(H,[H|F],L,F,L). % destiva(H,[1,2,3,4|L],L,FR,LR).
```

% 40. Implementati o stiva cu efecte laterale

```
adauga_stiva(X):-asserta(X).  
remove_stiva(X):-retract(nr(X)).
```

% 41. Implementati o coada cu efecte laterale

```
adauga_coadă(X):-assertz(X). %adauga_coadă(nr(4)).  
remove_coadă(X):-retract(nr(X)). %remove_coadă(X).
```

% 42. Adancimea unei liste adanci

maxim2nr(A,B,A):-A>B, !.

maxim2nr(A,B,B).

adancime_lista_adanca([],1).

adancime_lista_adanca([H|T],R):-atomic(H),!,

adancime_lista_adanca(T,R).

adancime_lista_adanca([H|T],R):-adancime_lista_adanca(H,R1),

adancime_lista_adanca(T,R2),

R3 is R1+1,

maxim(R3,R2,R).

% 43. (Aplatizare) flatten pe o lista adanca

flatten([],[]).

flatten([H|T],[H|R]):-atomic(H),!,

flatten(T,R).

flatten([H|T],R):-flatten(H,R1),

flatten(T,R2),

append(R1,R2,R).

% 44. (Aplatizare) flatten pe o lista adanca, incompleta

```
flatten_li(T,T):-var(T),!.
```

```
flatten_li([H|T],[H|R]):-atomic(H),!,
```

```
    flatten_li(T,R).
```

```
flatten_li([H|T],R):-flatten_li(H,R1),
```

```
    flatten_li(T,R2),
```

```
    append_li(R1,R2,R).
```

% 45. Inlocuirea unui element cu un alt element intr-o lista adanca

```
replace_lista_adanca(Old,New,[Old|T],[New|R]):-replace_lista_adanca(Old,New,T,R).
```

```
replace_lista_adanca(Old,New,[H|T],[H|R]):-atomic(H),!,
```

```
    replace_lista_adanca(Old,New,T,R).
```

```
replace_lista_adanca(Old,New,[H|T],R):-replace_lista_adanca(Old,New,H,R1),
```

```
    replace_lista_adanca(Old,New,T,R2),
```

```
    append([R1],R2,R).
```

```
replace_lista_adanca(_,_,[],[]).
```

```
member(X,[X|_]).
```

```
member(X,_|T):-member(X,T).
```

%46. Ce se afiseaza la introducerea member(X,[1,2,3,4,7])?

% Prima data se afiseaza X=1, dupa repetarea intrebarii se afiseaza

% X=2, X=3...

%47. Maximul dintr-o lista stabila si explicarea a ceea ce ar trebui facut

% pentru a deveni instabila

%stabil-ia prima aparitie

%instabil, o ia pe a doua

%.....

% 48. Se da o lista de arbori binari de cautare.

% a. Sa se determine arborele care are cele mai multe noduri cu 2 succesori

% b. Sa se determine arborele care are cea mai mica cheie

% c. Lista frunzelor din arbori(varianta cu append si cu liste diferenta)

% a. Sa se determine arborele care are cele mai multe noduri cu 2 succesori

nr_noduri_2succ(t(K,nil,nil),0).

nr_noduri_2succ(t(K,L,nil),N):-nr_noduri_2succ(L,N).

nr_noduri_2succ(t(K,nil,R),N):-nr_noduri_2succ(R,N).

nr_noduri_2succ(t(K,L,R),N):-nr_noduri_2succ(L,N1),

nr_noduri_2succ(R,N2),

N3 is N1+N2,

N is N3 +1.

```
max_arb([],Maxs,Maxs,MaxT,MaxT).
```

```
max_arb([H|T],Maxs,Maxsp,MaxT,MaxTP):-nr_noduri_2succ(H,Nr),
```

```
    Nr>Maxsp,!,
```

```
    max_arb(T,Maxs,Nr,MaxT,H).
```

```
max_arb([H|T],Maxs,Maxsp,MaxT,MaxTP):-max_arb(T,Maxs,Maxsp,MaxT,MaxTP).
```

```
max_arbore([H|T],Maxs,R):-max_arb([H|T],Maxs,0,R,nil).
```

% b. Sa se determine arborele care are cea mai mica cheie

```
cheie_minima(nil,Min,Min).
```

```
cheie_minima(t(K,L,_),Min,Minp):-K<Minp,!,
```

```
    cheie_minima(L,Min,K).
```

```
cheie_minima(t(K,L,_),Min,Minp):-cheie_minima(L,Min,Minp).
```

```
cheie_min(t(K,L,_),Min):-cheie_minima(L,Min,K).
```

```
min_cheie_arbore([],Maxs,Maxs,MaxT,MaxT).
```

```
min_cheie_arbore([H|T],Maxs,Maxsp,MaxT,MaxTP):-cheie_min(H,Nr),
```

```
    Nr<Maxsp,!,
```

```
    min_cheie_arbore(T,Maxs,Nr,MaxT,H).
```

```
min_cheie_arbore([H|T],Maxs,Maxsp,MaxT,MaxTP):-min_cheie_arbore(T,Maxs,Maxsp,MaxT,MaxTP).
```

```
min_cheie_arbore([H|T],Maxs,R):-min_cheie_arbore([H|T],Maxs,100,R,nil).
```

```
% c. Lista frunzelor din arbori(varianta cu append si cu liste diferenta)
```

```
colectare_frunze(nil,[]).
```

```
colectare_frunze(t(K,nil,nil),[K]).
```

```
colectare_frunze(t(K,L,R),Rez):-colectare_frunze(L,R1),  
                                colectare_frunze(R,R2),  
                                append(R1,R2,Rez).
```

```
colectare_total([],[]).
```

```
colectare_total([H|T],Rez):-colectare_frunze(H,R1),  
                            colectare_total(T,Rez1),  
                            append(Rez1,R1,Rez).
```

```
%colectare_total([],[]).
```

```
%colectare_total([H|T],[R1|Rez]):-colectare_frunze(H,R1),
```

```
%                                colectare_total(T,Rez).
```

```
%varianta cu liste diferenta
```

```
colectare_frunza_dl(nil,L,L).
```

```

colectare_frunza_dl(t(K,nil,nil),[K|LS],LS).

colectare_frunza_dl(t(K,L,R),LS,LE):-colectare_frunza_dl(L,LS,LT),
                                     colectare_frunza_dl(R,LT,LE).

%varianta fara append

colectare_total_ld1([],L,L).

colectare_total_ld1([H|T],Ls,Le):-colectare_frunza_dl(H,Ls,Last1),
                                   colectare_total_ld1(T,Last1,Le).

%varianta cu append

colectare_total_ld([],L,L).

colectare_total_ld([H|T],Ls,Le):-colectare_frunza_dl(H,First,Last),
                                   colectare_total_ld(T,First1,Last1),
                                   append_dif(First,Last,First1,Last1,Ls,Le).

%.....

% 49. lista de arbori incompleti

% a. Gasirea primului element atomic din structura

% b. Concatenarea tuturor elementelor din structura

% c. Arborele de diametru maxim

% a. Gasirea primului element atomic din structura

am_atom(T,_):-var(T),!,fail.

am_atom(t(K,_,_),K):-atomic(K),!.

am_atom(t(K,L,_),A):-am_atom(L,A).

```

```
am_atom(t(K,_,R),A):-am_atom(R,A).
```

```
afisare_atom([],0).
```

```
afisare_atom([H|T],A):-am_atom(H,A).
```

```
afisare_atom([H|T],A):-afisare_atom(T,A).
```

```
% c. Arborele de diametru maxim
```

```
inaltime_arbore(T,0):-var(T),!.
```

```
inaltime_arbore(t(_,L,R),Rez):-inaltime_arbore(L,R1),  
                                inalttime_arbore(R,R2),  
                                maxim2nr(R1,R2,R3),  
                                Rez is R3+1.
```

```
diametru_arbore(T,-1):-var(T),!.
```

```
diametru_arbore(t(_,L,R),D):-diametru_arbore(L,D1),  
                                diametru_arbore(R,D2),  
                                inalttime_arbore(L,H1),  
                                inalttime_arbore(R,H2),  
                                H3 is H1+H2,  
                                maxim2nr(D1,D2,DF),  
                                maxim2nr(DF,H3,D).
```

```
diametru_maxim([],Maxs,Maxs,MaxT,MaxT).
```

```
diametru_maxim([H|T],Maxs,Maxsp,MaxT,MaxTP):-diametru_arbore(H,Nr,
```

```
    Nr>Maxsp,!,
```

```
    diametru_maxim(T,Maxs,Nr,MaxT,H).
```

```
diametru_maxim([H|T],Maxs,Maxsp,MaxT,MaxTP):-diametru_maxim(T,Maxs,Maxsp,MaxT,MaxTP).
```

```
diametru_maxim_final([H|T],Maxs,R):-diametru_maxim([H|T],Maxs,0,R,nil).
```

```
%.....
```

```
% 50 Se da un graf in care fiecare muchie are o culoare.Determinati
```

```
%lista nodurilor incidente in muchii de o anumita culoare.
```

```
edge(1,2,rosu).
```

```
edge(3,4,verde).
```

```
edge(5,6,rosu).
```

```
edge(7,8,rosu).
```

```
collect(R,Rez):-retract(culoare(X)),
```

```
        X \=end,  
        collect([X|R],Rez).  
collect(Rez,Rez).
```

```
gasire_nod_culoare(C):-asserta(culoare(end)),  
        edge(A,B,C),  
        verifica(A),  
        verifica(B),  
        fail.
```

```
gasire_nod_culoare(_).
```

```
verifica(X):-culoare(X),!.  
verifica(X):-asserta(culoare(X)).
```

```
creare_lista(C,Rez):-gasire_nod_culoare(C),  
        collect([],Rez).
```

```
edge(a, b).  
edge(b, c).  
edge(c, d).  
edge(a, d).  
edge(a, c).  
edge(b, d).
```

```
neighbor(a,[b,c,d]).
```

```
neighbor(b, [a, c, d]).
```

```
neighbor(c, [b, d]).
```

```
neighb_to_edge:-retract(neighbor(Node,List)),!,
```

```
    process(Node,List),
```

```
    neighb_to_edge.
```

```
neighb_to_edge.
```

```
process(Node, [H|T]):-assertz(edge(Node, H)),
```

```
    process(Node, T).
```

```
process(_, []).
```

```
% 62. Se da reprezentarea de edge si ce cere transformare in neighbor
```

```
%edge(1,2).
```

```
%edge(1,3).
```

```
%edge(3,5).
```

```
%edge(4,6).
```

```
%edge(5,7).
```

```
muchie(X,Y):-edge(X,Y);edge(Y,X).
```

```
edge_to_neighbor:-
```

```
asserta(neighbor(end,[])),
```

```
muchie(X,_),
```

```
not(neighbor(X,_)),
```

```
findall(Y,muchie(X,Y),L),
```

```
assertz(neighbor(X,L)),
```

```
fail.
```

```
edge_to_neighbor.
```

```
collect_all_neighb(ACC,Rez):-retract(neighbor(A,B)),A\=end,collect_all_neighb([[A,B]|ACC],Rez).
```

```
collect_all_neighb(ACC,ACC).
```

```
afis_neighbor(Rez):-edge_to_neighbor,collect_all_neighb([],Rez).
```

% 63. Se da reprezentarea de neighbor si ce cere transformare in edge

```
neighbor(1,[2,3]).
```

```
neighbor(2,[1]).
```

```
neighbor(3,[1,5]).
```

```
neighbor(4,[6]).
```

```
neighbor(5,[3,7]).
```

```
neighbor(6,[4]).
```

```
neighbor(7,[5]).
```

```
neighb_to_edge:-asserta(edge(end,end)),
```

```
neighbor(X,L),
```

```
not(edge(X,_)),
```

```
process(X,L),
```

```
fail.
```

```
neighb_to_edge.
```

```
process(X,[H|T]):-assertz(edge(X,H)),process(X,T).
```

```
process(_,[]).
```

```
collect_all_edge(ACC,R):-retract(edge(A,B)),A\=end,collect_all_edge([edge(A,B)|ACC],R).
```

```
collect_all_edge(ACC,ACC).
```

```
collec_edge(R):-neighb_to_edge,collect_all_edge([],R).
```

```
%.....
```

```
% maxim stabil
```

```
maxim([],0).
```

```
maxim([H],H):-!.
```

```
maxim([H|T],H):-maxim(T,M), H>=M,!.
```

```
maxim([H|T],M):-maxim(T,M).
```

```
% Instabil daca inlocuim ">=" cu ">"
```

```
%exemplu graf
```

```
arc(2,1).
```

```
arc(1,2).
```

```
arc(3,1).
```

```
gr_int(X,R):- asserta(interior(0)),
```

```
            gr_int2(X),
```

```
            retract(interior(R)).
```

```
gr_int2(X):-arc(_,X),
```

```
            retract(interior(Y)),
```

NewY is $Y + 1$,

asserta(interior(NewY)),

fail.

gr_int2(_).